2-1 · 로컬 Kubernetes 배포 상태

> 2주차 실습 산출물 · **실행일 2026-10-18** (10/15 결과는 `_backup_2026-10-15/`에 보관) 클러스터 `three-tier` (kind v0.30.0) · WSL2 Ubuntu · Docker 29.2.1

이 문서의 모든 출력은 2026-10-18 19:33~19:45에 실제 실행한 결과다. 클러스터를 **삭제 후 처음부터 다시 생성**해서 얻었다.

---

1. 환경

항목	값
실행 위치	WSL2 Ubuntu (Windows 11)
컨테이너 런타임	Docker (WSL 안의 `dockerd`, kind 노드가 Docker 컨테이너로 동작)
kind / kubectl	v0.30.0 / v1.37.1
Kubernetes	v1.34.0 (`kindest/node:v1.34.0`)
노드	control-plane 1 + worker 2 (전부 Ready)
CNI	kindnet `kindest/kindnetd:v20250512-df8de77b`

```
kind delete cluster --name three-tier
kind create cluster --config k8s/kind-config.yaml
kubectl apply -f k8s/
```

> `kubectl apply -f k8s/`는 `kind-config.yaml`에서 `no matches for kind "Cluster"` 오류를 낸다. kind 설정 파일이 같은 폴더에 있기 때문이며 무해하다. 나머지 매니페스트는 모두 적용됐다. 오류를 피하려면 `00-`~`40-` 파일을 개별로 적용한다.

---

2. `kubectl get pods,svc` 결과 (모든 파드 Running)

web (namespace `web`)

```
pod/web-5b96fc5475-czz6q   1/1   Running   0   10.244.3.2   three-tier-worker
pod/web-5b96fc5475-l782f   1/1   Running   0   10.244.2.2   three-tier-worker2

service/web   NodePort   10.96.60.32   <none>   80:30080/TCP
```

api (namespace `api`)

```
pod/api-6bb5845668-8b44z   1/1   Running   0   10.244.2.3   three-tier-worker2
pod/api-6bb5845668-pt84p   1/1   Running   0   10.244.3.3   three-tier-worker

service/api   ClusterIP   10.96.1.99   <none>   8080/TCP
```

db (namespace `db`)

```
pod/db-0   1/1   Running   0   10.244.3.5   three-tier-worker

service/db   ClusterIP (Headless)   None   <none>   5432/TCP
```

판정

항목	기대	실제	판정
앱 파드 수	5 (web 2 · api 2 · db 1)	5	✅
STATUS	전부 `Running`	전부 `Running`	✅
READY	전부 `1/1`	전부 `1/1`	✅
RESTARTS	0	0	✅
web Service	`NodePort`	`NodePort` 80:30080	✅
api · db Service	외부 타입 아님	`ClusterIP` · Headless	✅
`tier` 라벨	전 파드	web / api / db 라벨 확인	✅

3. 연결성 검증 (실측)

시험	명령	결과
외부 → web	`curl http://localhost:8080`	**HTTP 200**, `web tier (nginx)` 페이지
외부 → api (호스트 포트)	`curl http://localhost:8081`	연결 실패 (매핑 없음)
외부 → db (호스트 포트)	`/dev/tcp/127.0.0.1/5432`	Connection refused (매핑 없음)
web → api	`wget http://api.api.svc.cluster.local:8080`	`{"tier":"api","runtime":"node v20.20.2",...}`
api → db	node `net.connect(5432)`	**CONNECTED**

시험	명령	결과
web → db	`nc -z -w4 db.db.svc.cluster.local 5432`	**BLOCKED** (rc=1)

4. NetworkPolicy 실제 강제 여부 — 대조 실험

`web → db` 차단이 정책 때문인지, 다른 원인(`nc` 부재 등)인지 구분하려고 정책을 껐다 켜 봤다. `nc`는 web 이미지의 BusyBox(`/usr/bin/nc`)에 있다.

단계	상태	web → db:5432
1	`db-allow-from-api` 정책 **적용**	BLOCKED
2	정책 **삭제** (`kubectl delete netpol`)	**CONNECTED**
3	정책 **재적용**	BLOCKED

정책이 있을 때만 막히고 없으면 뚫린다. **이 클러스터의 kindnet은 NetworkPolicy를 실제로 강제한다.**

> ⚠️ 10/15 기록 정정
>
> 10/15 문서는 "kind 기본 CNI(kindnet)는 NetworkPolicy를 강제하지 않으며 Calico로 교체해야 한다"고 적었다. 이번 실측으로 **그 서술은 틀렸다**. 최신 kindnet(`v20250512`)은 NetworkPolicy를 구현한다. 따라서 Calico 교체는 필요 없고, AC-5는 로컬에서도 완전 충족이다. (10/15에 실제로 시험하지 않고 알려진 통념으로 적은 것이 원인이다.)

5. 재현 방법

```
# WSL2 Ubuntu 안에서
export PATH="$HOME/.local/bin:$PATH"
cd "/mnt/d/02.study/매치업 AI 강의/실습산출물/2주차"

kind create cluster --config k8s/kind-config.yaml
kubectl wait --for=condition=Ready nodes --all --timeout=180s
for f in k8s/0*.yaml k8s/[1-4]0-*.yaml; do kubectl apply -f $f; done
kubectl wait --for=condition=Ready pods --all -A --timeout=300s
for n in web api db; do kubectl get pods,svc -n $n; done
curl -s http://localhost:8080
```

정리: `kind delete cluster --name three-tier`

6. 운영 메모

- WSL은 **연결된 프로세스가 없으면 종료**되어 kind 노드 컨테이너가 같이 죽는다. 다시 켜면 파드가 `Unknown`으로 보이다가 1~3분 뒤 회복한다. 상태 확인은 WSL을 켜 둔 채 한 번에 실행하는 스크립트로 한다.
- 이번 세션에서 WSL을 여러 번 껐다 켠 탓에 이전 클러스터의 RESTARTS가 올라갔다. 위 결과는 **새로 만든 클러스터**의 값이라 RESTARTS=0이다.

7. Docker Hub 이미지로 재배포 (2026-10-18 19:52)

web·api를 직접 빌드해 Docker Hub(`jjang404`)에 올리고, 매니페스트가 그 이미지를 pull하도록 바꿨다. (ConfigMap으로 소스를 주입하던 방식 제거) 링크·digest: docker/README.md

계층	파드가 실행 중인 이미지 (imageID)
web	`docker.io/jjang404/matchup-w2-web:1.1 (멀티 아키텍처, digest 0ab10d5b…)`
api	`docker.io/jjang404/matchup-w2-api:1.1 (멀티 아키텍처, digest 96aa9d7d…)`
db	`docker.io/library/postgres@sha256:721873c3…` (공식 이미지)

재배포 후 재검증: web·api 파드 `Running 1/1`, `localhost:8080` = 200, web→api 응답, api→db CONNECTED, web→db BLOCKED.

8. Web → WAS(api) → DB 전 구간 검증 (2026-10-18)

api 1.1에 `/dbcheck`(TCP 5432 접속 확인), web 1.1에 `/api/` 프록시를 추가해 한 번의 요청으로 세 계층을 통과시켰다. (WSL이 반복 재시작되어 이전 클러스터의 인증서가 손상돼 클러스터를 한 번 더 새로 만들었고, 아래는 그 새 클러스터의 결과다.)

#	검증	결과
1	파드 5개	전부 `Running 1/1`, RESTARTS 0
3	DB 기동	`pg_isready` → accepting connections (PostgreSQL 16.15)
4	DB 읽기·쓰기	임시 테이블 create/insert/select 성공 (`1:web-was-db`)
5	WAS → DB	`/dbcheck` → `db_reachable:true`
6	WEB → WAS	web 파드에서 `127.0.0.1/api/` → api JSON 응답
7	**외부 → WEB → WAS → DB**	`curl localhost:8080/api/dbcheck` → **HTTP 200** `db_reachable:true`
9	WEB → DB 직접	**BLOCKED** (NetworkPolicy)
10	대조: db 접근 차단	`/api/dbcheck` → **HTTP 503** `db_reachable:false`
11	복구	정책 원복 후 HTTP 200

> `wget localhost`는 IPv6(`::1`)로 풀려 `Connection refused`가 난다 (nginx는 IPv4만 listen). 시험 주소의 문제이며 `127.0.0.1`은 정상이다. 한계: `/dbcheck`는 TCP 도달성까지다. SQL 실행은 `db-0` 안에서 별도로 확인했다.